

Laboratory 10:

Sampling Theorem, Reconstruction, & Aliasing Sampling and Reconstruction Demonstrator

From Continuous to Discrete and Back: Building Real Hardware

Lab Duration: 2.5 hours

Pre-Lab Time Expected: 40 minutes (Nyquist theorem calculations and aliasing predictions)

Key Concept: A continuous-time signal can be perfectly reconstructed from equally-spaced samples *if and only if* the sampling rate exceeds twice the highest signal frequency. This lab brings the Nyquist-Shannon theorem to life: you will sample analog signals at various rates, observe aliasing artifacts, reconstruct with analog filters, and prove that theory matches hardware measurements.

Contents

1	Real-World Context: Sampling Across Signal Acquisition Systems	4
1.1	Program 1: High-Speed Digital Storage Oscilloscopes (Test & Measurement)	4
1.2	Program 2: High-Fidelity Audio Recording Systems (Music & Entertainment)	4
1.3	Program 3: Medical Sensor Data Acquisition (Biomedical Devices)	5
1.4	The Unifying Principle: Nyquist, Reconstruction, and Aliasing Prevention	5
2	Learning Objectives	6
3	Core Concepts: Sampling, Nyquist, and Reconstruction	7
3.1	The Nyquist-Shannon Sampling Theorem	7
3.2	Alias Frequencies	7
3.3	Reconstruction from Samples	7
4	Required Equipment and Software	8
5	Safety and Wiring Notes	9
6	Pre-Lab Preparation (Complete Before Lab Session)	10
6.1	Nyquist Frequency and Alias Prediction	10
6.2	RC Filter Design	11
7	Hardware Setup: Sample-and-Hold Circuit	12
7.1	CD4066 Analog Switch Basics	12
7.2	Wiring Configuration	12
8	Lab Procedure (In-Lab Execution, 2.5 Hours)	14
8.1	Part 1: Pre-Lab Check and Timer Configuration	14
8.2	Part 2: Sampling at Multiple Rates	14
8.3	Part 3: Reconstruction with RC Filter	15
8.4	Part 4: MATLAB Acquisition and FFT Analysis	15

8.5	Part 5: Simulink Modeling and Validation	16
9	Post-Lab Analysis and Deliverables	18
9.1	Required Discussion Questions	18
9.2	Final Submission Checklist	19
9.3	TA Checkpoint Sign-Off	20
	Appendix A: Nyquist-Shannon Theorem (Reference)	21
	Appendix B: CD4066 Analog Switch and Timer Configuration	22
	Appendix C: Simulink Sampling and Reconstruction Template	23
	References and Inspiration	24

Grading and Authenticity Requirements

Deliverable (1/2): This is a graded laboratory assignment. Include your **name and student ID** on all handwritten work, oscilloscope screenshots, circuit photos, and MATLAB plots.

Deliverable (2/2): Your hand-derived Nyquist frequency predictions and alias frequency calculations must be shown to the TA before hardware measurements. Using online calculators is not accepted; you must compute these by hand from the Nyquist theorem.

Checkpoint (1/6): Before final submission, verify that all oscilloscope waveforms are clearly labeled with timebase, voltage scale, and your name/ID. Sampled and reconstructed waveforms must be shown side-by-side for comparison.

1 Real-World Context: Sampling Across Signal Acquisition Systems

You are a Junior Signal Acquisition Engineer at a company designing data acquisition and test equipment. Your expertise in the Nyquist-Shannon sampling theorem is critical to three very different product lines, each handling signals at vastly different frequencies and with different consequences for aliasing errors.

1.1 Program 1: High-Speed Digital Storage Oscilloscopes (Test & Measurement)

Your company manufactures digital oscilloscopes used by hardware engineers to debug circuits, validate prototypes, and characterize signal integrity in digital systems operating at gigahertz speeds.

The challenge: An oscilloscope must capture fast transients (microsecond-scale details) while also showing the overall signal envelope (millisecond or longer). The sampling rate determines what you can observe. Too slow, and fast edges appear blurry and aliased. Too fast, and you waste memory and storage bandwidth.

The engineering problem: A digital circuit operates at 1 GHz. To observe timing relationships and metastability events, you need to sample at $\geq 2 \times 1 \text{ GHz} = 2 \text{ GHz}$ (Nyquist minimum). But actually, professional oscilloscopes use $\geq 5 - 10\times$ the signal bandwidth to get clean eye diagrams and characterize jitter. Your job: design the sample-and-hold amplifier that feeds the analog-to-digital converter. Calculate Nyquist limits, predict alias frequencies if sampling is inadequate, and measure the analog reconstruction filter's cutoff frequency to prevent aliasing from entering the acquisition chain.

Regulatory requirement: Prove that your oscilloscope's sample-and-hold circuit introduces < 5 ns of delay variation (aperture jitter) across the temperature range, and that the reconstruction filter has ≥ 80 dB attenuation at the Nyquist frequency to suppress aliases that would corrupt measurements.

1.2 Program 2: High-Fidelity Audio Recording Systems (Music & Entertainment)

Your company manufactures digital audio interfaces for professional recording studios, concert venues, and broadcast stations. Recording engineers demand transparent, artifact-free audio capture.

The challenge: Human hearing spans approximately 20 Hz to 20 kHz. Professional audio standards use 44.1 kHz (CD), 48 kHz (video/film), or 96 kHz (mastering) sampling rates. The standard 44.1 kHz was chosen specifically to exceed the Nyquist rate ($2 \times 20 \text{ kHz} = 40 \text{ kHz}$). But at exactly 40 kHz Nyquist, any signal energy at 20 kHz aliases back to 0 Hz (DC). Real signals extend beyond 20 kHz (ultrasonic transients from instruments). A sharp anti-aliasing filter is critical.

The engineering problem: Design an anti-aliasing filter that passes 0–20 kHz transparently (with no phase distortion or group delay variation) while attenuating everything above 22 kHz by > 90 dB. This requires a high-order filter (4th–8th order butterworth or Chebyshev). Your lab models this filtering in Simulink and validates it empirically by sampling signals at various rates and listening for artifacts.

Quality requirement: Measure the alias rejection ratio: if you intentionally apply a 25 kHz tone (which should alias to 19 kHz under 44.1 kHz sampling), quantify how much energy appears at the alias frequency in the reconstructed signal. Specification: < -80 dB alias suppression (inaudible to trained ears).

1.3 Program 3: Medical Sensor Data Acquisition (Biomedical Devices)

Your company manufactures wearable and implantable biomedical sensors: ECG monitors, glucose meters, motion sensors, and respiratory monitors. Each sensor samples at a rate determined by the biological signal's bandwidth, power budget, and regulatory requirements.

The challenge: Heart rate (ECG) spans 0.5–100 Hz. Respiratory rate is 0.1–0.5 Hz. Motion (accelerometer) spans 0 Hz (static tilt) to ~ 10 kHz (vibration). A glucose meter samples blood glucose once every 5 minutes. Each signal requires a different sampling strategy. Undersample and you miss events; oversample and you waste power (every sample drains the battery).

The engineering problem: Design a multi-rate sampling strategy for a wearable health monitor: ECG at 250 Hz (Nyquist for 100 Hz content, plus margin), motion at 100 Hz (sufficient for fall detection), glucose at 0.1 Hz (once per 10 minutes). Each signal must be anti-aliased before downsampling. Your lab implements a sample-and-hold circuit and demonstrates alias rejection. You'll measure how much motion artifact (noise) aliases into the ECG band when sampling too slowly.

Clinical requirement: Prove that your anti-aliasing filter preserves clinically relevant ECG features (ST-segment details, T-wave morphology) that occur at 10–50 Hz, while rejecting motion noise (artifacts above 200 Hz that would alias into the ECG band under slow sampling).

1.4 The Unifying Principle: Nyquist, Reconstruction, and Aliasing Prevention

All three programs depend on mastering the same fundamental concepts:

1. **The Sampling Theorem:** If a signal is band-limited to frequency $f_{\max}$, you must sample at $f_s \geq 2f_{\max}$ (Nyquist rate) to preserve all information. Undersample, and frequency components above $f_s/2$ alias down into the passband, corrupting the signal irreversibly.
2. **Aliasing Prediction:** Hand-calculate where alias frequencies will appear. A signal at f_0 sampled at f_s produces aliases at $|kf_s \pm f_0|$ for integer k .
3. **Anti-Aliasing Filter Design:** Place an analog low-pass filter (cutoff at $f_s/2$ or slightly lower) before the sampler to remove high-frequency content that would alias.
4. **Reconstruction from Samples:** After digitization and processing, a digital-to-analog converter with a reconstruction filter returns the signal to continuous-time, approximating the original analog waveform.
5. **Hardware Validation:** Build the sample-and-hold circuit, measure its performance, compare empirical results to Nyquist predictions, and document any deviations (due to real component tolerances, settling time, and noise).

In this lab, you will build a sample-and-hold circuit controlled by an Arduino timer, sample analog signals at various rates, reconstruct with an RC low-pass filter, and prove that your measurements match the Nyquist-Shannon theorem.

2 Learning Objectives

By the end of this lab, you should be able to:

1. **Understand the Nyquist-Shannon sampling theorem:** Explain why sampling at twice the highest signal frequency is necessary and sufficient to preserve all information in a band-limited signal.
2. **Predict alias frequencies by hand:** Given a signal frequency and sampling rate, calculate where alias frequencies will appear in the sampled spectrum and sketch the result.
3. **Design sample-and-hold circuits:** Build a practical circuit using an analog switch (CD4066) and capacitor that captures and holds a voltage sample for accurate digitization.
4. **Control sampling with Arduino timers:** Configure an Arduino timer to generate precise, periodic sampling pulses at specified rates (adjustable from 1 kHz to 100 kHz).
5. **Observe aliasing in real hardware:** Sample a signal below the Nyquist rate and observe alias tones in the oscilloscope and FFT—proof that undersampling corrupts the signal.
6. **Design and measure reconstruction filters:** Hand-calculate RC time constants for analog low-pass filters and measure their frequency response via oscilloscope sweep.
7. **Validate theory with Simulink:** Build an ideal sampler and reconstruction filter in Simulink, overlay the simulated output against hardware measurements, and quantify agreement.
8. **Understand real-world imperfections:** Explain how finite aperture time, sample-and-hold droop, filter rolloff, and quantization noise cause practical systems to deviate from the ideal Nyquist-Shannon model.

3 Core Concepts: Sampling, Nyquist, and Reconstruction

3.1 The Nyquist-Shannon Sampling Theorem

A continuous-time signal $x(t)$ that contains no frequency components higher than $f_{\max}$ (i.e., is *band-limited*) can be completely reconstructed from a set of samples taken at equal intervals, provided the sampling rate f_s satisfies:

$$f_s \geq 2f_{\max}. \quad (1)$$

The minimum rate $f_s = 2f_{\max}$ is called the **Nyquist rate**. The quantity $f_{\text{Nyquist}} = f_s/2$ is the **Nyquist frequency**—the highest frequency that can be represented without ambiguity at a given sampling rate.

Consequence of undersampling: If $f_s < 2f_{\max}$, frequency components above $f_s/2$ are *aliased*—they appear as lower-frequency components in the sampled signal, and the original signal cannot be reconstructed.

3.2 Alias Frequencies

When a continuous-time signal $x(t)$ containing a sinusoid at frequency f_0 is sampled at rate f_s , the discrete-time samples contain not only the original frequency but also *aliases* at frequencies:

$$f_{\text{alias}} = |kf_s \pm f_0|, \quad k = 0, 1, 2, \dots \quad (2)$$

Example: A 25 kHz sinusoid sampled at 44.1 kHz produces aliases at:

$$|1 \times 44.1 - 25| = 19.1 \text{ kHz}, \quad (3)$$

$$|2 \times 44.1 - 25| = 63.2 \text{ kHz}, \quad (4)$$

$$|0 \times 44.1 + 25| = 25 \text{ kHz}, \quad (\text{original, above Nyquist}). \quad (5)$$

The 19.1 kHz alias is in the audible range and will be heard if not filtered out.

3.3 Reconstruction from Samples

An ideal reconstructor uses the *sinc interpolation* formula:

$$x(t) = \sum_{n=-\infty}^{\infty} x[n] \text{sinc}(\pi f_s(t - nT_s)), \quad (6)$$

where $x[n]$ are the sample values and $T_s = 1/f_s$ is the sample period.

In practice, a simple low-pass RC filter approximates this sinc interpolation, smoothing between sample values.

[INSERT DIAGRAM: Time-domain sampling of sinusoid, spectrum showing aliases, reconstruction filter response]

4 Required Equipment and Software

Table 1. Required Hardware and Software

Category	Required Items
Controller	Arduino Uno or Mega, USB cable
Analog sampling	CD4066 quad analog switch (CMOS), capacitor 100 nF, op-amp buffer (TL072 or LM358)
Signal generation	Function generator (0.1 Hz to 100 kHz), or Arduino PWM/DAC
Measurement	Oscilloscope (100 MHz bandwidth minimum), 2 channels
Reconstruction	Resistor (10 k Ω), capacitor (10 nF to 1 μ F, tunable)
Prototyping	Breadboard, jumper wires, potentiometer for adjustable filter cutoff
Software	Arduino IDE, MATLAB R2021a or later, Simulink
Provided skeletons	<code>SampleAndHold.ino</code> , <code>Lab10_Sampling.m</code> , <code>Lab10_Reconstruction.slx</code>

Arduino Timer Configuration

The Arduino's Timer1 (16-bit) will be configured to generate periodic interrupt signals that control the CD4066 sampling switch. The sampling frequency is set by the Timer1 overflow rate:

$$f_s = \frac{f_{\text{clock}}}{N \times (\text{TOP} + 1)}, \quad (7)$$

where $f_{\text{clock}} = 16$ MHz, N is the prescaler, and TOP is the timer's maximum count value.

Student Notes: Timer1 configuration parameters:

Prescaler: _____ (1, 8, 64, 256, or 1024)

TOP value: _____ (0 to 65535)

Target f_s : _____ Hz

5 Safety and Wiring Notes

- **High-frequency signals:** Function generators operating above 100 kHz can couple into nearby circuits and create ground loops. Keep signal wires short and twisted.
- **Breadboard contact resistance:** At high frequencies (>10 kHz), contact resistance in breadboards can affect filter response. Use quality jumper wires and ensure solid connections.
- **Capacitor leakage current:** The sample-and-hold capacitor must have low leakage (use film or quality ceramic; avoid cheap electrolytics). Leakage causes the held voltage to decay between samples.
- **Op-amp settling:** Op-amp buffer following the sample-and-hold must settle quickly (< 1 μ s for frequencies above 100 kHz). Use high-slew-rate op-amps (TL072, LM358, or faster).
- **Ground connections:** All grounds (Arduino, function generator, oscilloscope, breadboard) must be connected to a common point to avoid ground loops and noise coupling.

6 Pre-Lab Preparation (Complete Before Lab Session)

6.1 Nyquist Frequency and Alias Prediction

Pre-lab Deliverable (1/2): For each test signal frequency assigned by the TA, hand-calculate the Nyquist frequency for several sampling rates and predict where aliases will appear.

Nyquist Theorem Application

For a sinusoidal signal at frequency f_0 , the Nyquist rate is:

$$f_s^{\min} = 2f_0. \quad (8)$$

The Nyquist frequency (maximum representable frequency) at sampling rate f_s is:

$$f_{\text{Nyquist}} = \frac{f_s}{2}. \quad (9)$$

Student Notes: Pre-lab calculations:

Signal frequency assigned by TA: $f_0 =$ _____ Hz

Minimum Nyquist sampling rate: $f_s^{\min} = 2 \times f_0 =$ _____ Hz

For $f_s = 10$ kHz: Nyquist frequency = _____ Hz

For $f_s = 5$ kHz: Nyquist frequency = _____ Hz

For $f_s = 2$ kHz: Nyquist frequency = _____ Hz

Alias Frequency Prediction

Using the formula $f_{\text{alias}} = |kf_s \pm f_0|$, calculate where your signal will alias under each sampling rate:

Student Notes: Predicted alias frequencies:At $f_s = 10$ kHz and $f_0 =$ _____ Hz:

Primary alias: _____ Hz (this should be observable on the oscilloscope)

At $f_s = 5$ kHz and $f_0 =$ _____ Hz:

Primary alias: _____ Hz

At $f_s = 2$ kHz and $f_0 =$ _____ Hz:

Primary alias: _____ Hz

6.2 RC Filter Design

For reconstruction, you will use a simple RC low-pass filter. Calculate the cutoff frequency for a given R and C:

$$f_c = \frac{1}{2\pi RC}. \quad (10)$$

Choose R and C values such that f_c is at or slightly below the Nyquist frequency ($f_s/2$):

Student Notes: Filter design:Target Nyquist frequency (for $f_s = 10$ kHz): $f_c =$ _____ HzChoose $R = 10$ k Ω . Then $C = \frac{1}{2\pi f_c R} =$ _____ F(Practical capacitor value: _____ nF or μ F)

Checkpoint (2/6): Show the TA your handwritten Nyquist predictions and alias calculations for Checkpoint 1 sign-off before hardware measurements begin.

7 Hardware Setup: Sample-and-Hold Circuit

7.1 CD4066 Analog Switch Basics

The CD4066 is a CMOS quad bilateral switch—when a digital control signal is HIGH, it closes the switch (low resistance path); when LOW, it opens (high impedance). This allows the Arduino to connect/disconnect a signal path at precise timing intervals.

For sampling:

- Input: analog signal from function generator.
- Control: digital signal from Arduino Timer1 interrupt (HIGH for $\sim 1\ \mu\text{s}$ to sample, LOW to hold).
- Output (through capacitor): holds the sampled voltage.
- Buffer: op-amp follows to provide low-impedance output and prevent capacitor discharge.

7.2 Wiring Configuration

1. **Function generator** output $\rightarrow$ resistor ($10\ \text{k}\Omega$) $\rightarrow$ CD4066 input pin (pin 1 for switch A).
2. **CD4066 output** (pin 2) $\rightarrow$ capacitor ($100\ \text{nF}$) to ground. This capacitor holds the sample.
3. **Capacitor node** $\rightarrow$ non-inverting input of op-amp buffer (TL072 or LM358).
4. **Op-amp** (configured as unity-gain buffer): output is the sampled signal, low impedance.
5. **CD4066 control pin** (pin 13) $\leftarrow$ Arduino Timer1 output (pin 11 on Uno, PWM or OC1A).
6. **CD4066 power**: pins 14 (V_{DD} , +5V), pins 7 & 8 (V_{SS} , GND).
7. **Oscilloscope**: Channel 1 = function generator input. Channel 2 = op-amp output (sampled signal).

[INSERT CIRCUIT SCHEMATIC: Function generator $\rightarrow$ resistor $\rightarrow$ CD4066 (analog switch) $\rightarrow$ capacitor $\rightarrow$ TL072 buffer $\rightarrow$ scope and reconstruction RC filter]

Student Notes: Wiring checklist:

- ☐ Function generator output connected to CD4066 input via 10 k Ω resistor
- ☐ 100 nF sample capacitor from CD4066 output to GND
- ☐ Op-amp buffer (unity gain) following capacitor
- ☐ Arduino Timer1 control signal to CD4066 switch pin

Checkpoint (8/6) and ground connections: **TA verified** all connections, verifies capacitor is installed, and checks that Timer1 output is connected to the CD4066 control pin.

- ☐ RC reconstruction filter connected to op-amp output

8 Lab Procedure (In-Lab Execution, 2.5 Hours)

Target Timeline: CP-1 (15 min) → Hardware setup (20 min) → Sampling at various rates (40 min) → Reconstruction testing (30 min) → Simulink validation (30 min) → wrap-up (5 min).

8.1 Part 1: Pre-Lab Check and Timer Configuration

1. Show the TA your handwritten Nyquist and alias predictions.
2. Power on the Arduino and verify Timer1 is outputting a periodic signal on pin 11 (oscilloscope).
3. Measure the Timer1 output frequency and verify it matches your target sampling rate.
4. Power on the function generator and set it to your assigned signal frequency f_0 .

Checkpoint (4/6): TA verifies Nyquist calculations and confirms Timer1 frequency matches expected f_s .

8.2 Part 2: Sampling at Multiple Rates

Objective: Observe the effect of sampling rate on the captured waveform.

1. Set the function generator to frequency f_0 (assigned by TA).
2. Configure Arduino Timer1 for sampling rate $f_s = 50$ kHz (well above Nyquist).
3. Capture the sampled signal on the oscilloscope. Take a photograph.
4. Reduce the sampling rate to $f_s = 10$ kHz. Observe and photograph the reconstructed waveform.
5. Further reduce to $f_s = 5$ kHz (below Nyquist if $f_0 > 2.5$ kHz). Observe aliasing artifacts. Photograph.
6. Further reduce to $f_s = 2$ kHz. Aliasing becomes very apparent. Photograph.

Student Notes: Observations at each sampling rate:

$f_s = 50$ kHz: Signal looks clean. Frequency observed: _____ Hz

$f_s = 10$ kHz: Some aliasing visible. Frequency observed: _____ Hz

$f_s = 5$ kHz: Significant aliasing. Alias frequency: _____ Hz (compare to prediction)

$f_s = 2$ kHz: Heavy aliasing. Alias frequency: _____ Hz

Checkpoint (5/6): TA verifies that you have oscilloscope photos at a minimum of 3 sampling rates. **Checkpoint 2 (CP-2):** Compare observed alias frequencies to your pre-lab predictions.

8.3 Part 3: Reconstruction with RC Filter

Objective: Demonstrate that an analog low-pass filter can suppress aliases and reconstruct a smooth waveform.

1. Connect the RC reconstruction filter to the output of the sample-and-hold op-amp buffer.
2. Set the function generator to f_0 and sampling rate to $f_s = 5$ kHz (below Nyquist for $f_0 > 2.5$ kHz).
3. With no filter (or very high cutoff), observe the sampled signal on Channel 2. It will show alias artifacts.
4. Insert the RC filter you designed in pre-lab. Observe the output. The waveform should smooth out.
5. Measure the filter's frequency response by sweeping the function generator from 1 kHz to 20 kHz (if $f_s = 5$ kHz, cutoff should be near 2.5 kHz). Record the amplitude attenuation at each frequency.

**[INSERT EXAMPLE OSCILLOSCOPE PLOT: Sampled
(aliased) signal vs. reconstructed (filtered) signal
side-by-side]**

Student Notes: Reconstruction filter measurements:

Designed cutoff frequency: _____ Hz

Measured -3dB frequency: _____ Hz (use scope to find where amplitude drops to 70.7%)

Alias suppression at $f_s/2$: _____ dB

Checkpoint (6/6): TA verifies that the reconstructed waveform shows alias suppression compared to the unfiltered sampled signal.

8.4 Part 4: MATLAB Acquisition and FFT Analysis

Objective: Capture the sampled signal into MATLAB and verify alias frequencies using FFT.

1. Set function generator to f_0 and sampling rate to $f_s = 5$ kHz.

2. Use the Arduino to stream the sampled values (via Serial port) at 115200 baud.
3. In MATLAB, read the serial data and reconstruct the sampled signal vector.
4. Compute the FFT of the captured signal.
5. Plot the magnitude spectrum and identify the peak frequencies.
6. Compare to your hand-calculated alias predictions.

[INSERT EXAMPLE MATLAB PLOT: FFT magnitude spectrum showing original frequency (if $f_0 < f_s/2$) and aliases]

Student Notes: FFT results:

Peak frequency 1: _____ Hz (Expected: _____ Hz)

Peak frequency 2: _____ Hz (Expected alias: _____ Hz)

Error in alias frequency: _____ %

8.5 Part 5: Simulink Modeling and Validation

Objective: Build an ideal sampler and reconstruction filter in Simulink and compare to hardware.

1. Open the provided `Lab10.Reconstruction.slx` Simulink model.
2. Set the input signal frequency to f_0 (your assigned value).
3. Set the sampling rate to $f_s = 5$ kHz.
4. Set the reconstruction filter cutoff to match your measured f_c .
5. Run the simulation and plot the output.
6. Overlay the Simulink output against your measured hardware data (from MATLAB).
7. Quantify the error: RMS difference between Simulink and hardware.

**[INSERT EXAMPLE MATLAB PLOT: Hardware sampled
signal overlaid with Simulink ideal reconstruction]**

Student Notes: Simulink validation:

RMS error between Simulink and hardware: _____ mV or %

Discrepancies noted: _____

9 Post-Lab Analysis and Deliverables

9.1 Required Discussion Questions

Pre-lab Deliverable (2/2): Answer each question in 3–6 complete sentences, supported by numerical evidence from your lab measurements.

1. **Nyquist Theorem Validation:** For your assigned signal frequency, what was the minimum sampling rate you observed to capture the signal without aliasing? How does this compare to the theoretical Nyquist rate $2f_0$? Discuss any discrepancies.
2. **Alias Characterization:** When you sampled at $f_s = 5$ kHz, you observed alias frequencies. Hand-calculate where these aliases should appear, compare to your measurements, and explain the match (or mismatch).
3. **Reconstruction Filter Effectiveness:** Measure the alias suppression (in dB) provided by your RC filter. Is this sufficient to make the alias inaudible or invisible on the scope? Justify with calculations or observations.
4. **Real-World vs. Ideal:** Your Simulink model assumes perfect sampling (instantaneous, noiseless) and ideal reconstruction (sinc interpolation). Your hardware has finite aperture time, quantization noise, and RC filter rolloff. How do these real-world effects appear in your measurements?
5. **Application Trade-offs:** For the three applications discussed (oscilloscope, audio recording, biomedical sensing), explain why each chose its particular sampling rate. Why don't all systems just use the highest possible sampling rate?
6. **System Design Specification:** You are designing a new wearable ECG monitor. Heart-rate diagnostically relevant content is 0.5–100 Hz. Propose a sampling rate, design the anti-aliasing filter, and justify your choices based on Nyquist theorem and your lab observations.

Student Notes: Space for discussion answers (or attach separate sheet with name/ID):

9.2 Final Submission Checklist

Before submitting, verify that your submission includes **all of the following**:

- ☐ **Pre-lab calculations:** Handwritten Nyquist frequency and alias predictions for all test sampling rates, with name/ID.
- ☐ **Circuit diagram:** Photo or sketch of your breadboard setup showing sample-and-hold circuit, CD4066, capacitor, and op-amp buffer.
- ☐ **Oscilloscope photos:** 4+ waveform screenshots showing sampled signal at different sampling rates ($f_s = 50, 10, 5, 2$ kHz or similar), with timebase and voltage scales labeled and your name/ID visible.
- ☐ **Reconstruction comparison:** Side-by-side oscilloscope photos of unfiltered (aliased) and filtered (reconstructed) sampled signal, clearly labeled.
- ☐ **MATLAB Figure 1:** Original (function generator) input signal waveform, with axis labels, legend, and your name/ID in title.
- ☐ **MATLAB Figure 2:** Sampled signal (at $f_s = 5$ kHz or below Nyquist), showing visible aliasing, with axis labels and name/ID.

- ☐ **MATLAB Figure 3:** FFT magnitude spectrum of sampled signal, clearly identifying the original frequency and alias peaks with vertical annotations and name/ID in title.
- ☐ **MATLAB Figure 4:** Reconstruction filter frequency response (measured via oscilloscope sweep or simulated). Show magnitude response from 1 Hz to 10 kHz with -3 dB point marked.
- ☐ **Simulink validation plot:** Hardware-measured sampled signal overlaid with Simulink ideal reconstruction. Legend must distinguish the two. Include RMS error annotation.
- ☐ **Filter design summary:** Printed table showing: designed R, designed C, target cutoff frequency, measured cutoff frequency, and discrepancy.
- ☐ **Discussion answers:** Written responses to six discussion questions with numerical support and your name/ID.
- ☐ **Arduino code appendix:** Print key sections of `SampleAndHold.ino` showing Timer1 configuration and sampling rate calculation.
- ☐ **MATLAB code appendix:** Print the section that reads serial data, computes FFT, and plots the spectrum.
- ☐ **TA sign-off page:** See Table 2 below, completed with TA initials at each checkpoint.

9.3 TA Checkpoint Sign-Off

Table 2. TA Checkpoint Sign-Off (Complete as Lab Progresses)

CP	Requirement	Target Time	TA Initials & Date
CP-1	Pre-lab Nyquist and alias predictions verified. Timer1 frequency checked and confirmed.	0:00–0:15	_____
CP-2	Sampled waveforms captured at 4+ rates. Observed alias frequencies match hand predictions. Oscilloscope photos taken with scales labeled.	0:15–0:55	_____
CP-3	RC reconstruction filter installed and tested. Frequency response measured or simulated. Reconstructed waveform shows alias suppression.	0:55–1:40	_____
CP-4	MATLAB FFT analysis complete. Alias frequencies identified in spectrum. Simulink model overlaid with hardware data.	1:40–2:30	_____

Lab Complete: Once all four checkpoints are signed off and the final submission checklist is complete, your lab is done. Congratulations on validating the Nyquist-Shannon sampling theorem in real hardware!

Appendix A: Nyquist-Shannon Theorem (Reference)

A.1 Mathematical Statement

Let $x(t)$ be a signal whose frequency content is limited to frequencies below $f_{\max}$ (i.e., $X(f) = 0$ for $|f| > f_{\max}$, where $X(f)$ is the Fourier transform of $x(t)$).

If samples are taken at a rate $f_s > 2f_{\max}$, then $x(t)$ can be uniquely recovered from the sample sequence $\{x(nT_s)\}_{n=-\infty}^{\infty}$, where $T_s = 1/f_s$.

A.2 Aliasing Under Undersampling

If $f_s < 2f_{\max}$, the sampled signal's discrete-time Fourier transform exhibits *aliasing*: frequency components above $f_s/2$ are reflected back into the band $[0, f_s/2]$ and are indistinguishable from lower-frequency components.

For a sinusoid at frequency f_0 , the aliases appear at $|kf_s \pm f_0|$ for integer k .

A.3 Anti-Aliasing and Reconstruction

- **Anti-aliasing:** Before sampling, apply a low-pass filter with cutoff at $f_s/2$ to remove frequency components that would alias.
- **Reconstruction:** After digital processing, use a reconstruction filter (cutoff at $f_s/2$) and DAC to return the signal to continuous time.
- **Ideal reconstruction filter:** The sinc function $\text{sinc}(x) = \sin(\pi x)/(\pi x)$ is the ideal interpolation kernel, achieving perfect reconstruction when the Nyquist condition is met.

Appendix B: CD4066 Analog Switch and Timer Configuration

B.1 CD4066 Pin Configuration

The CD4066 is a quad bilateral switch. For sampling, use switch A:

- Pin 1: Common (input from signal source).
- Pin 2: Normally-open contact (output to capacitor).
- Pin 13: Control input (HIGH to close switch, LOW to open).
- Pins 14, 7, 8: Power (V_{DD} , V_{SS} , V_{SS}).

When Pin 13 is HIGH (control signal asserted), the switch closes (ON-resistance $\approx 100\ \Omega$). When LOW, the switch opens (OFF-resistance $\approx 10\ \text{M}\Omega$).

B.2 Arduino Timer1 Configuration for Sampling

To generate a periodic sampling pulse, configure Timer1 (16-bit) in CTC (Clear Timer on Compare) mode:

```
// Configure Timer1 for 10 kHz sampling (16 MHz clock, prescaler 8)
// f_s = 16,000,000 / (8 * 200) = 10,000 Hz
TCCR1A = 0x00;           // Normal mode
TCCR1B = 0x0A;           // CTC mode, prescaler = 8
OCR1A = 199;             // Compare value (TOP = 199, counts 0-199 = 200 ticks)
TIMSK1 = 0x02;           // Enable compare A interrupt

ISR(TIMER1_COMPA_vect) {
    digitalWrite(11, HIGH); // Trigger sample pulse
    delayMicroseconds(1);    // Hold for ~1  $\mu$ s
    digitalWrite(11, LOW);   // Release
}
```

Adjust OCR1A to change sampling rate: $f_s = 16,000,000 / (8 \times (\text{OCR1A} + 1))$.

B.3 Sample-and-Hold Timing

The CD4066 control signal must be a narrow pulse (1–10 μ s width) that occurs at regular intervals (period = $1/f_s$). The pulse width determines the effective sampling duration. Narrower pulses reduce droop but require faster slew-rate amplifiers to settle the capacitor voltage.

Appendix C: Simulink Sampling and Reconstruction Template

C.1 Ideal Sampler Block

In Simulink, use the "Discrete-Time Integrator" or "Unit Delay" block to model sampling. Alternatively, configure a "Sample Time Specification" in a Subsystem to enforce $T_s = 1/f_s$.

```
% Set sample time to enforce Ts = 1/fs
% In Simulink block dialog: Sample Time = [1/fs 0]
```

C.2 Reconstruction Filter in Simulink

Use the "Transfer Fcn" block with a first-order low-pass filter:

$$H(s) = \frac{2\pi f_c}{s + 2\pi f_c}, \quad (11)$$

where $f_c = 1/(2\pi RC)$ is the cutoff frequency.

In Simulink numerator/denominator form:

```
Numerator: [2*pi*fc]
Denominator: [1, 2*pi*fc]
```

C.3 Zero-Order Hold (Practical Reconstruction)

Between samples, a zero-order hold maintains the last sampled value until the next sample arrives. In Simulink, use the "Zero-Order Hold" block with sample time = $1/f_s$.

C.4 Plotting Hardware vs. Simulink

```
% Load hardware data from MATLAB (from serial read)
hardware_time = (0:length(hardware_signal)-1) / fs;
hardware_signal = ...; % Vector of sampled values

% Simulink output is already in the workspace (from Scope block)
simulink_time = tout;
simulink_signal = yout;

% Plot overlay
figure;
plot(hardware_time, hardware_signal, 'b-', 'LineWidth', 1.5, 'DisplayName', 'Hardware');
hold on;
plot(simulink_time, simulink_signal, 'r--', 'LineWidth', 1.5, 'DisplayName', 'Simulink');
xlabel('Time (s)'); ylabel('Amplitude (V)');
title(sprintf('Your Name (ID) - Hardware vs. Simulink Reconstruction'));
legend; grid on;
```

References and Inspiration

This laboratory manual draws inspiration from decades of signal processing pedagogy, hardware engineering practice, and real-world system design across multiple industries. The following sources informed the curriculum design, real-world context narratives, and technical content.

Foundational Theory and Pedagogy

- **Nyquist-Shannon Sampling Theorem:** H. Nyquist, “Certain Factors Affecting Telegraph Speed,” *Bell System Technical Journal*, vol. 3, pp. 324–346, 1924; and C. E. Shannon, “Communication in the Presence of Noise,” *Proceedings of the IRE*, vol. 37, no. 1, pp. 10–21, 1949.
- **Digital Signal Processing Foundations:** A. V. Oppenheim and R. W. Schaffer, *Discrete-Time Signal Processing*, 3rd ed. Pearson, 2010. Chapter 4 covers sampling and reconstruction in depth.
- **Laboratory Design for Signal Processing Education:** J. H. McClellan, R. W. Schaffer, and M. A. Yoder, *DSP First: A Multimedia Approach*, 2nd ed. Pearson, 2015. This text emphasizes hands-on learning and intuitive understanding of sampling.
- **Hardware-Software Co-Design in Education:** Numerous engineering programs (MIT, UC Berkeley, Georgia Tech, and others) have pioneered the integration of embedded systems with signal processing theory, demonstrating that laboratory validation of mathematical models significantly improves student understanding.

Real-World Industry Applications and Standards

The Real-World Context narratives in this lab draw from established engineering practice in three domains:

Test & Measurement (Digital Oscilloscopes)

- **Oscilloscope Design Standards:** IEEE 1057-2007, “IEEE Standard for Digitizing Waveform Recorders,” establishes sampling rate requirements and timing accuracy specifications for real-time and equivalent-time sampling.
- **Signal Integrity Engineering:** E. Bogatin, *Signal and Power Integrity - Simplified*, 2nd ed. Prentice Hall, 2009. Discusses how sampling rate and anti-aliasing filters affect measurements of high-speed digital signals.
- **Professional Equipment References:** Tektronix, Keysight, Rigol, and other oscilloscope manufacturers document their sample-and-hold circuits, aperture jitter specifications, and anti-aliasing filter designs in published technical notes and application guides.

Audio Engineering (High-Fidelity Recording)

- **Audio Sampling Standards:** The 44.1 kHz CD standard (Redbook Audio, 1980) and 48 kHz video standard (ITU-R Rec. 601) are rooted in careful application of the Nyquist theorem. See K. C. Pohlmann, *Principles of Digital Audio*, 6th ed. McGraw-Hill, 2011.
- **Anti-Aliasing Filter Design:** J. G. Proakis and D. G. Manolakis, *Digital Signal Processing: Principles, Algorithms, and Applications*, 4th ed. Pearson, 2006. Chapter 9 covers filter design for anti-aliasing and reconstruction.

- **Professional Audio Practice:** Specifications from manufacturers like RME, Apogee, Universal Audio, and Zoom document real-world anti-aliasing and reconstruction filter characteristics, typically achieving 80–120 dB alias suppression.

Biomedical Device Design

- **ECG Sampling Standards:** American Heart Association (AHA) and International Electrotechnical Commission (IEC) standards specify sampling rates for ECG: typically 250–1000 Hz for clinical-grade monitors (IEC 60601-2-25 for ECG equipment).
- **Wearable Sensor Design:** B. Yildirim, A. Jain, F. Balli, and S. Erçil, “Wearable Sensor Design for Real-Time Health Monitoring,” in *Wearable Sensors: Fundamentals, Implementation and Applications*, ed. E. P. Enderle and J. D. Bronzino, Academic Press, 2014.
- **Power-Efficient Sampling:** Low-power sampling strategies and multi-rate filtering are documented in numerous papers on implantable medical devices (IMDs) and wearables in *IEEE Transactions on Biomedical Engineering* and *IEEE Journal of Biomedical and Health Informatics*.

Hardware and Software Tools

This laboratory employs widely-adopted open-source and industrial tools:

- **Arduino:** Open-source hardware and software platform widely used in education and embedded systems. Timer configurations follow official Arduino reference documentation.
- **MATLAB and Simulink:** Industry-standard tools for signal processing, control design, and system modeling. MathWorks documentation and numerous textbooks provide examples of discrete-time signal processing and filter design.
- **CD4066 Analog Switch:** CMOS bilateral switch manufactured by multiple vendors (Texas Instruments, ON Semiconductor, NXP, etc.). Datasheets and application notes provide guidance on sample-and-hold circuit design.

Pedagogical Philosophy and University Inspirations

This lab manual builds upon decades of innovative signal processing pedagogy from leading universities. The curriculum follows established best practices in engineering education:

- **Active Learning:** Hands-on experimentation with real hardware validates abstract mathematical concepts (Nyquist, aliasing, reconstruction).
- **Industry Relevance:** Real-world context narratives ground theory in concrete engineering challenges faced by professional engineers.
- **Multi-Domain Approach:** The same signal processing principles appear across test equipment, audio, and biomedical domains—students see transferable knowledge.
- **Verification and Validation:** Students compare hardware measurements to theoretical predictions (via Simulink), developing critical thinking and debugging skills essential for professional practice.

University Course Inspirations

The laboratory sequence draws explicit inspiration from three flagship undergraduate signal processing programs:

Georgia Tech ECE 2026: Real-Time DSP with Audio Processing

Georgia Tech’s signals and systems course emphasizes real-time audio processing on embedded hardware. This lab series adopts Georgia Tech’s philosophy of immediate auditory feedback (students hear aliasing artifacts, filter effects, synthesis results) to build intuition. The use of Arduino for real-time signal generation and acquisition, and the emphasis on convolution-based filtering (Labs 04, 09), reflect the hands-on DSP-on-microcontroller approach pioneered in ECE 2026.

MIT 6.003 / 6.007: Filters, Sampling, and Frequency-Domain Analysis

MIT’s foundational and advanced signal processing courses have long emphasized physical demonstrations of signal processing concepts. Lab 10 (Sampling Reconstruction) directly mirrors MIT 6.007’s classic sampling theorem demonstrations using op-amp circuits and real-time oscilloscope observation. MIT’s treatment of analog filter design (passive RC, active op-amp filters) informs Labs 03, 08, and 09. The emphasis on hand-calculation of filter specifications before hardware build reflects MIT’s pedagogical commitment to mathematical rigor paired with hardware validation.

Rice University ELEC 241: Real-Time Convolution and Filtering on Embedded Systems

Rice University’s course on real-time signal processing using TI LaunchPad microcontrollers inspired the convolution-heavy focus in Labs 04 and 09. Rice’s integration of Simulink modeling with embedded code generation (via Embedded Coder) shapes the Model-Based Design philosophy throughout this series. The emphasis on hand-deriving filter coefficients, building practical implementations, and comparing theoretical vs. empirical results comes directly from Rice’s laboratory pedagogy.

Acknowledgments

This lab manual was developed as part of a comprehensive curriculum redesign for undergraduate signal and systems education. The real-world context narratives and laboratory design reflect:

- **Academic Foundations:** Extensive review of signal processing curricula at Georgia Tech, MIT, Rice University, UC Berkeley, and Carnegie Mellon University.
- **Industry Experience:** Cumulative professional experience in software engineering, embedded systems, and DSP at technology companies (Apple, Google, Qualcomm, Microsoft, etc.).
- **Hardware Design and Validation:** Real-world engineering practice in aerospace, medical device, telecommunications, and consumer electronics sectors.
- **University-Industry Partnerships:** Feedback from practicing engineers and academic faculty on what skills students need to succeed in modern signal processing careers.

The curriculum emphasizes that signal processing is not an abstract mathematical discipline—it is the foundation of the technologies that power modern civilization: wireless communications, medical diagnostics, audio reproduction, and countless autonomous systems. By combining rigorous theory, hardware implementation, and real-world context, students develop both deep understanding and practical competence.